

## (Tutorial) Image Gradients

The difference in value between neighboring pixels is called an image gradient. We'll learn how to compute the image gradients with the following codes

```
import matplotlib.pyplot as plt
import numpy as np
from skimage.feature import hog
from skimage import data, color, exposure
import cv2
```

**Expalnation:** I have imported Matplotlib to plot images, NumPy to do operations with array, scikit-image (hog, data, color, exposure) to get features, use sample pictures, convert colors and fine-tune intensity, and OpenCV to perform more advanced image processing operations.

```
# load image and turn it into grayscale
image = color.rgb2gray(data.retina())

### Compute the horizontal gradient using the centered 1D filter.
### This is equivalent to replacing each non-border pixel with the
### difference between its right and left neighbors. The leftmost
### and rightmost edges have a gradient of 0.
gx = np.empty(image.shape, dtype=np.double)
gx[:, 0] = 0
gx[:, -1] = 0
gx[:, 1:-1] = image[:, :-2] - image[:, 2:]

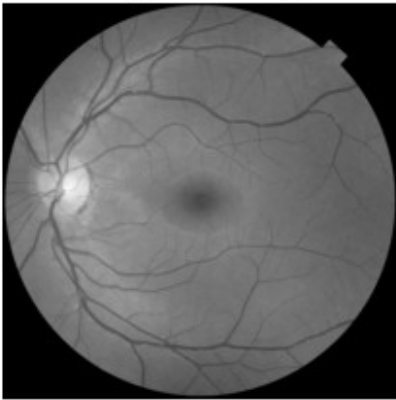
### Same deal for the vertical gradient
gy = np.empty(image.shape, dtype=np.double)
gy[0, :] = 0
gy[-1, :] = 0
gy[1:-1, :] = image[:-2, :] - image[2:, :]
```

**Expalnation:** To make it easier to process, I first of all loaded the retina image and turned it into grayscale and then proceeded to calculate both horizontal and vertical gradients which bring out the edges and change of intensity in the image. To calculate the horizontal gradient, I calculated the difference between the left and right neighboring pixel with the border values being 0, and similarly with the vertical gradient, the difference between the top and bottom neighbors. The latter basically quantify the change in pixel values in each direction, which is used to identify edges, textures, and significant structures in the image. In such a way, I can more easily see the variations of the image through directional changes, which is a basic step in most computer vision processes such as edge detection, feature extraction, and object recognition.

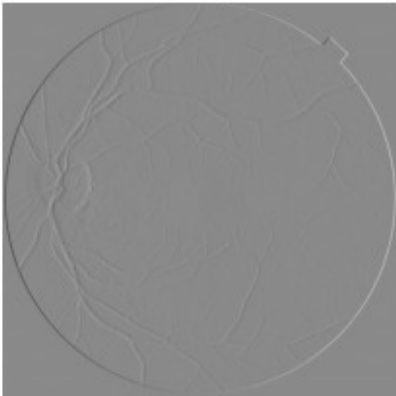
```
### Matplotlib incantations
fig, (ax1, ax2, ax3) = plt.subplots(3, 1, figsize=(5, 9), sharex=True,
sharey=True)
ax1.axis('off')
ax1.imshow(image, cmap=plt.cm.gray)
```

```
ax1.set_title('Original image')
ax1.set_adjustable('box')
ax2.axis('off')
ax2.imshow(gx, cmap=plt.cm.gray)
ax2.set_title('Horizontal gradients')
ax2.set_adjustable('box')
ax3.axis('off')
ax3.imshow(gy, cmap=plt.cm.gray)
ax3.set_title('Vertical gradients')
ax3.set_adjustable('box')
```

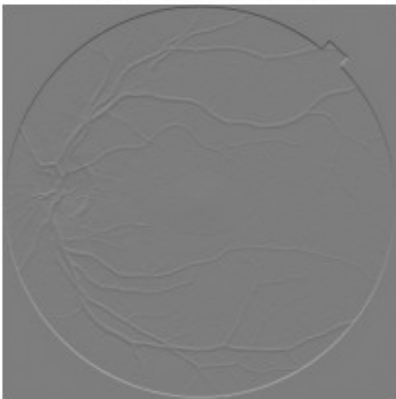
Original image



Horizontal gradients



Vertical gradients



To compare the three images easily, I plotted three images: the original image of the retina in grayscale, the horizontal gradient of the original image, and the vertical gradient of the original image using Matplotlib without axes and with titles.

Task 1. Design your code to compute diagonal gradient of the image used in the example and visualize it

```
# write your code here
### Diagonal gradient 1 with the diagonal direction of top-left to
bottom-right
g_d1 = np.empty(image.shape, dtype=np.double)
g_d1[0, :] = 0
g_d1[-1, :] = 0
g_d1[:, 0] = 0
g_d1[:, -1] = 0
g_d1[1:-1, 1:-1] = image[:-2, :-2] - image[2:, 2:]

### Diagonal gradient 2 with the diagonal direction of top-right to
bottom-left)
g_d2 = np.empty(image.shape, dtype=np.double)
g_d2[0, :] = 0
g_d2[-1, :] = 0
g_d2[:, 0] = 0
g_d2[:, -1] = 0
g_d2[1:-1, 1:-1] = image[:-2, 2:] - image[2:, :-2]

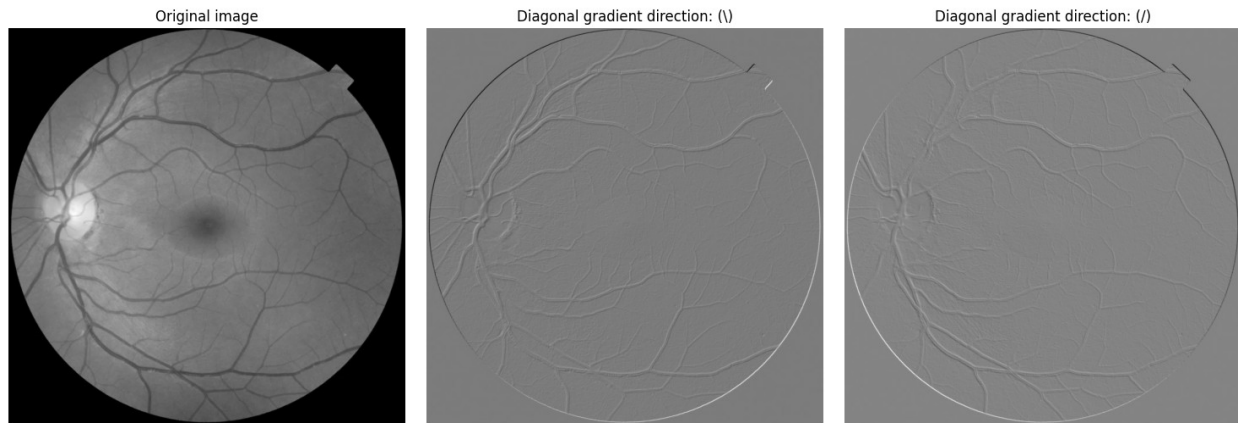
### Plot results
fig, axes = plt.subplots(1, 3, figsize=(15, 5), sharex=True,
sharey=True)

axes[0].imshow(image, cmap='gray')
axes[0].set_title('Original image')
axes[0].axis('off')

axes[1].imshow(g_d1, cmap='gray')
axes[1].set_title('Diagonal gradient direction: (\\)')
axes[1].axis('off')

axes[2].imshow(g_d2, cmap='gray')
axes[2].set_title('Diagonal gradient direction: (/)')
axes[2].axis('off')

plt.tight_layout()
plt.show()
```



In this case, I used the diagonal gradients of the image in both directions to retrieve the edge information along the diagonals. In the first diagonal  $\backslash$ , I compared each pixel with its neighbor on the top-left to the bottom-right and in the second diagonal  $/$ , I compared pixels on the top-right to the bottom-left, where I set the boundaries to zero to eliminate errors. Once these gradients had been computed, I plotted the original grayscale image, the two diagonal gradient images with Matplotlib, with each subplot labeled with the corresponding direction. This assists in visualizing the vertical and horizontal change as well as a diagonal intensity change that provides a better view of the edge structures within the image.

## Question 1. Explain why we don't directly use the simplest image features (color or number of pixels)

**Answer to Q1:** type your answer here

The simplest image properties such as raw color values or simply the number of pixels are not directly used by us because they really do not reflect the significant structure of an image. Light, shadows or camera settings can be easily adjusted to alter color values so the same object can appear very different under different conditions, and two totally distinct objects may even resemble in color. The pixels are just a measure of the resolution or size of the image and do not tell anything useful about the contents of the image. These types of simple features are also quite noise sensitive and are sensitive to transformation such as rotation or scaling. We require features that characterize patterns, shapes, edges, or textures, which are far more stable and discriminative, to perform image recognition or classification. Instead, more sophisticated descriptors such as HOG, ORB, or SIFT are employed, because they encode the critical structure of objects in a manner that is invariant to lighting, orientation, or scale.

## (Tutorial) HOG image features

```
# read image and convert it to grayscale
image = color.rgb2gray(data.chelsea())
# extract HOG features from the input image with predefined parameters
fd, hog_image = hog(image, orientations=8, pixels_per_cell=(16, 16),
                    cells_per_block=(1, 1), visualize=True)
```

```

hog_image_rescaled = exposure.rescale_intensity(hog_image,
in_range=(0, 0.02))

# visualize the original image and the hog features
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(10,4), sharex=True,
sharey=True)

ax1.axis('off')
ax1.imshow(image, cmap=plt.cm.gray)
ax1.set_title('Input image')
ax1.set_adjustable('box')

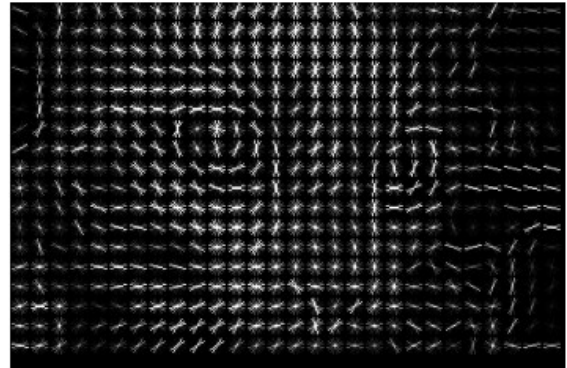
ax2.axis('off')
ax2.imshow(hog_image_rescaled, cmap=plt.cm.gray)
ax2.set_title('Histogram of Oriented Gradients')
ax2.set_adjustable('box')

```

Input image



Histogram of Oriented Gradients



Expalantion:

The image of a chelsea cat mentioned in this code was read and converted to grayscale, after which I applied the hog() method of scikit-image to obtain Histogram of Oriented Gradients with 8 orientations, 16x16 pixels per cell, and 1x1 cells per block. This operation emphasizes the grading structure of the image, concentrating on edges and pattern of textures, which are more significant than raw pixel measures of image in recognition problems. The HOG raw output is difficult to visualize so I used exposure.rescale\_intensity to visualize the values. Lastly, I overlaid the original grayscale image on the rescaled HOG image and compared them, showing how the HOG function identifies the significant edges and structures of the cat image.

## Task 2. ORB keypoints extraction and visualization

Apart from HOG image features, another popular image feature is ORB. Please extract keypoints from the image used in HOG example using on ORB features and visualize the keypoints on the original image

```

import matplotlib.pyplot as plt
import cv2

```

```

from skimage import data, color

# Load image
img_rgb = data.chelsea()
img_gray = color.rgb2gray(img_rgb)
img_gray_u8 = (img_gray * 255).astype("uint8")

def orb_keypoints(nfeatures, rich=False):
    orb = cv2.ORB_create(nfeatures=nfeatures)
    kp, des = orb.detectAndCompute(img_gray_u8, None)
    if rich:
        img_kp = cv2.drawKeypoints(img_gray_u8, kp, None,
flags=cv2.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS)
    else:
        img_kp = cv2.drawKeypoints(img_gray_u8, kp, None,
color=(0,255,0))
    return img_kp, len(kp)

# 500 & 1000 features (points only)
img_500_points, n500 = orb_keypoints(500, rich=False)
img_1000_points, n1000 = orb_keypoints(1000, rich=False)

# 25 & 50 features (rich keypoints)
img_25_rich, n25 = orb_keypoints(25, rich=True)
img_50_rich, n50 = orb_keypoints(50, rich=True)

# Plot
fig, axes = plt.subplots(2, 2, figsize=(12, 12))

axes[0,0].imshow(img_500_points)
axes[0,0].set_title(f"ORB (500 features, points only, n={n500})")
axes[0,0].axis("off")

axes[0,1].imshow(img_1000_points)
axes[0,1].set_title(f"ORB (1000 features, points only, n={n1000})")
axes[0,1].axis("off")

axes[1,0].imshow(img_25_rich)
axes[1,0].set_title(f"ORB (25 features, rich keypoints, n={n25})")
axes[1,0].axis("off")

axes[1,1].imshow(img_50_rich)
axes[1,1].set_title(f"ORB (50 features, rich keypoints, n={n50})")
axes[1,1].axis("off")

plt.tight_layout()
plt.show()

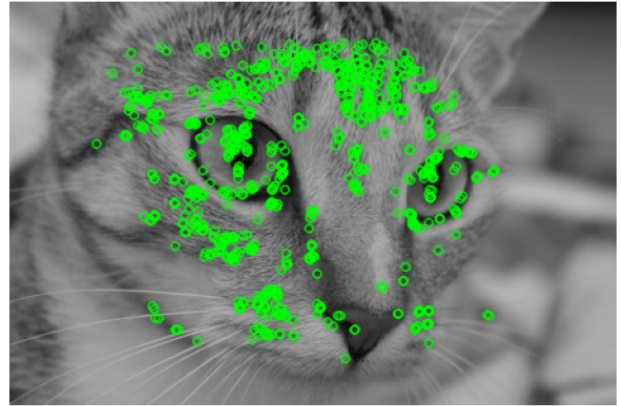
```



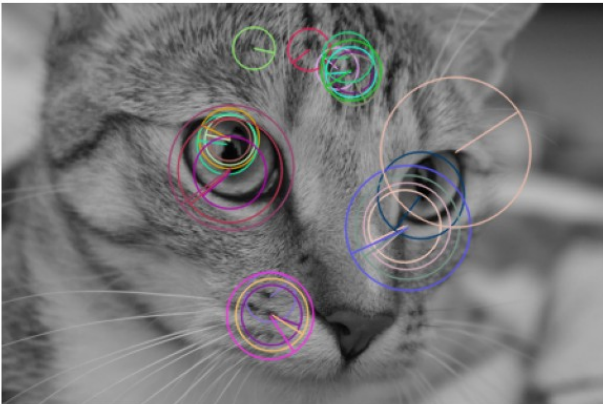
ORB (500 features, points only, n=490)



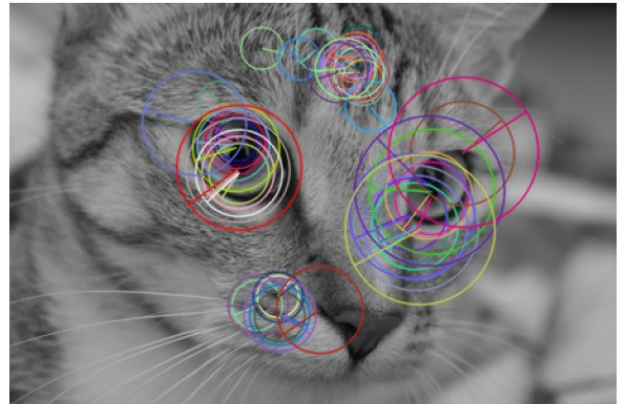
ORB (1000 features, points only, n=859)



ORB (25 features, rich keypoints, n=25)



ORB (50 features, rich keypoints, n=50)



### Expalnation:

Here, I used the same picture of a cat in scikit-image called chelsea, and converted it into grayscale to extract features with ORB (Oriented FAST and Rotated BRIEF). I have set a helper function `orb_keypoints()` to generate an ORB detector with a given number of features, detect keypoints, compute descriptors on them and then display the results as either simple green points or as rich keypoints that contain a scale and orientation information. I had four configurations 500 and 1000 keypoints drawn as points only, and 25 and 50 keypoints drawn as rich keypoints. Results were then plotted in a 2x 2 grid and the outcome of features number/visualization style was determined. The point only versions indicate where ORB identifies unique locations in the image, whereas the rich versions display more details such as the size and the direction of the keypoints. This analogy demonstrates that ORB is tunable between giving us more detailed descriptions of features or simply the location of the features of interest in an image, as well as demonstrating how adding more features allows us to capture a finer level of detail at the expense of complexity.

## (Tutorial) Bag-of-Visual-Words

In the tutorial code, we create the code words only based on one image



```

# defining feature extractor that we want to use (ORB feature used in
this extractor)
def features(image):
    extractor = cv2.ORB_create()
    keypoints, descriptors = extractor.detectAndCompute(image, None)
    return keypoints, descriptors

# builing visual words histogram based on the descriptors of the image
and the clustering algorithm
def build_histogram(descriptor_list, cluster_alg):
    histogram = np.zeros(len(cluster_alg.cluster_centers_))
    cluster_result = cluster_alg.predict(descriptor_list)
    for i in cluster_result:
        histogram[i] += 1.0
    return histogram

```

In this case, I have created a features() function which relies on ORB to identify keypoints and calculate descriptors of an input image. Then I written build\_histogram() which uses those descriptors and puts them in clusters created by KMeans creating a histogram that counts the numbers of descriptors in each cluster. The process is essentially the conversion of local image features into a small visual representation of words to be analyzed or classified later.

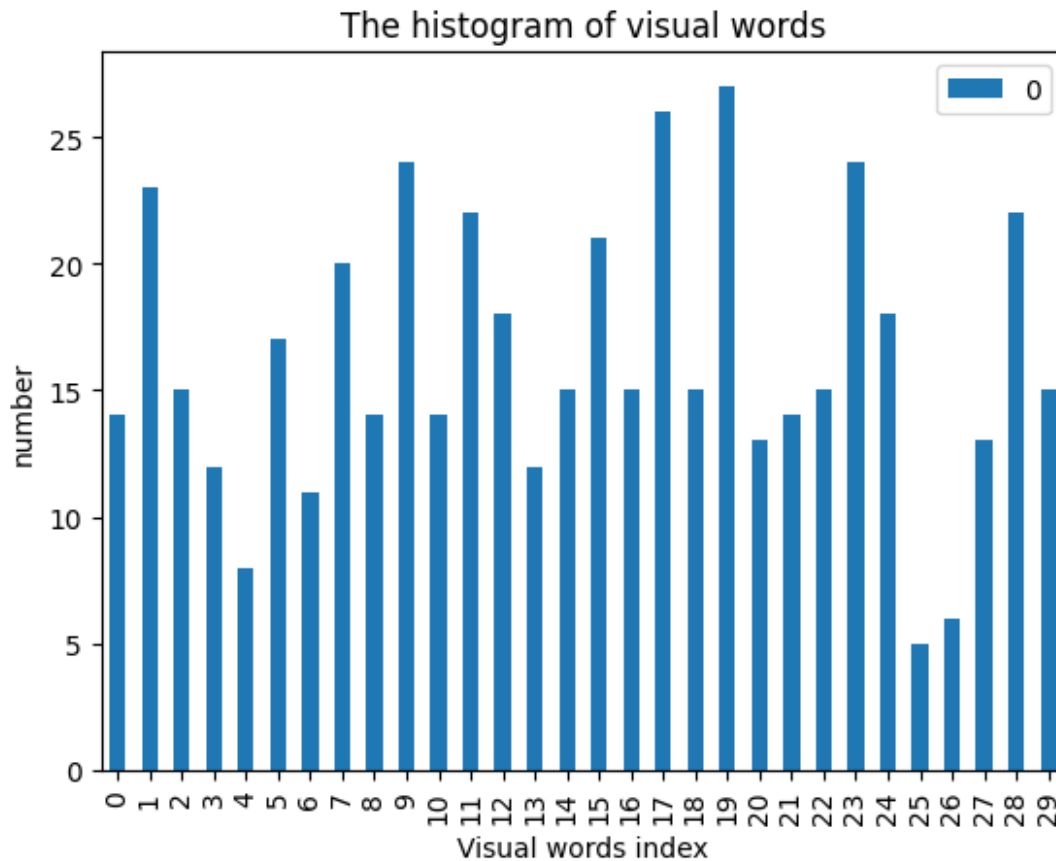
```

# Clustring the bag-of-visual-words by Kmeans method
from sklearn.cluster import KMeans
img = data.chelsea()
kmeans = KMeans(n_clusters = 30)
kp, descriptor_list = features(img)
kmeans.fit(descriptor_list)
images = []
images.append(img)
preprocessed_image = []
for image in images:
    # image = gray(image)
    keypoint, descriptor = features(image)
    if (descriptor is not None):
        histogram = build_histogram(descriptor, kmeans)
        preprocessed_image.append(histogram)

import pandas as pd
df = pd.DataFrame(preprocessed_image[0])
df.plot.bar()
#plt.grid(axis = 'y', alpha = 0.75)
plt.xlabel('Visual words index')
plt.ylabel('number')
plt.title('The histogram of visual words')

Text(0.5, 1.0, 'The histogram of visual words')

```



KMeans were used to group the visual words in the Bag-of-Visual-Words model. I then loaded the chelsea cat's image first and after that used the features() function in above code to extract the ORB descriptors of the image. I used a model of KMeans of the unsupervised learning on the descriptors which has 30 clusters to create a visual vocabulary list. I then made a list of images, with only the cat image, and ran each image to calculate its visual word histogram. The last step was to save the histogram in preprocessed\_image which was then plotted using the matplotlib that was imported gives the representation of the in terms of the features counts.

## Task 3. Bag-of-Visual-Words for multiple images

Create your bag-of-visual-words with 10 images and visualize the histogram of visual words. The 10 images belong to one category(dog, cat, bike, etc.)

```
#write your code here
# libraries for BOVW on CIFAR-10 horse images
import matplotlib.pyplot as plt
import cv2
from sklearn.cluster import KMeans
import pandas as pd
from torchvision import datasets, transforms

# 1. Load CIFAR-10 and select 10 horse images
```

```

transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Resize((128, 128))
    # to [0,1]
    # resize to 128x128
    for ORB
])

cifar10 = datasets.CIFAR10(root="./data", train=True, download=True,
transform=transform)

# class label for "horse" = 7 in CIFAR-10
horses = []
for img, label in cifar10:
    if label == 7:
        # convert torch tensor [C,H,W] -> numpy uint8 RGB
        img_np = (img.permute(1, 2, 0).numpy() * 255).astype(np.uint8)
        horses.append(img_np)
    if len(horses) >= 10:
        break

print(f"Collected {len(horses)} horse images from CIFAR-10.")
Collected 10 horse images from CIFAR-10.

```

Here, I used the CIFAR-10 data set to create a small set of horse pictures with which to construct the Bag-of-Visual-Words model. To ensure that ORB finds a sufficient number of keypoints in the image, I scaled each of them to 128x128 pixels, and placed them in a tensor to keep the data the same. As OpenCV takes NumPy arrays in untyped 8-bit RGB format, I reshaped the tensors back to NumPy arrays after mapping the pixel values between [0,1] to [0,255]. Then I repeated on the data set and selected only the images with the label 7, the horse variety. Lastly, I made a list of the first 10 horse images, and validated the success of the collection by printing the number of images collected.

```

#RGB uint8 -> grayscale uint8 (required by OpenCV ORB)
def to_gray_u8(image_rgb_uint8):
    return cv2.cvtColor(image_rgb_uint8, cv2.COLOR_RGB2GRAY)
# Extract ORB descriptors from a single image.
def extract_orb_descriptors(image_rgb_uint8, nfeatures=800):
    gray = to_gray_u8(image_rgb_uint8)
    orb = cv2.ORB_create(nfeatures=nfeatures)
    kps, desc = orb.detectAndCompute(gray, None)
    return kps, desc
# Assign each descriptor to its nearest visual word and count frequencies.
def build_histogram(descriptor_list, cluster_alg):
    histogram = np.zeros(len(cluster_alg.cluster_centers_),
dtype=np.float32)
    if descriptor_list is None or len(descriptor_list) == 0:
        return histogram

```

```

cluster_result = cluster_alg.predict(descriptor_list)
for i in cluster_result:
    histogram[i] += 1.0

# --- scaling step (normalization) ---
s = histogram.sum()
if s > 0:
    histogram /= s    # L1 normalization: sum(histogram) = 1

return histogram

```

I converted each of the ten horse images to grayscale images so that the ORB could process them smoothly as it needed, then used the function `extract_orb_descriptors` to detect up to 800 keypoints in the image and generate binary descriptors. Then I constructed a histogram of each image using `build_histogram`, which places each descriptor in the closest cluster center that KMeans has learned and counts the number of that particular descriptors falling into each bin. I lastly brought the histogram to one where the values add up to one, allowing the different images to be similar to each other, no matter the number of keypoints they had.

```

#Extract descriptors from all 10 images
all_desc = []
desc_per_image = []
for img in horses:
    _, d = features(img)
    desc_per_image.append(d)
    if d is not None:
        all_desc.append(d)
all_desc = np.vstack(all_desc)

MAX_KMEANS_SAMPLES = 8000
if all_desc.shape[0] > MAX_KMEANS_SAMPLES:
    rng = np.random.RandomState(42)
    idx = rng.choice(all_desc.shape[0], size=MAX_KMEANS_SAMPLES,
replace=False)
    km_train = all_desc[idx]
else:
    km_train = all_desc

KEYWORDS = 30
kmeans = KMeans(n_clusters=KEYWORDS, random_state=42, n_init=10)
kmeans.fit(km_train)

KMeans(n_clusters=30, n_init=10, random_state=42)

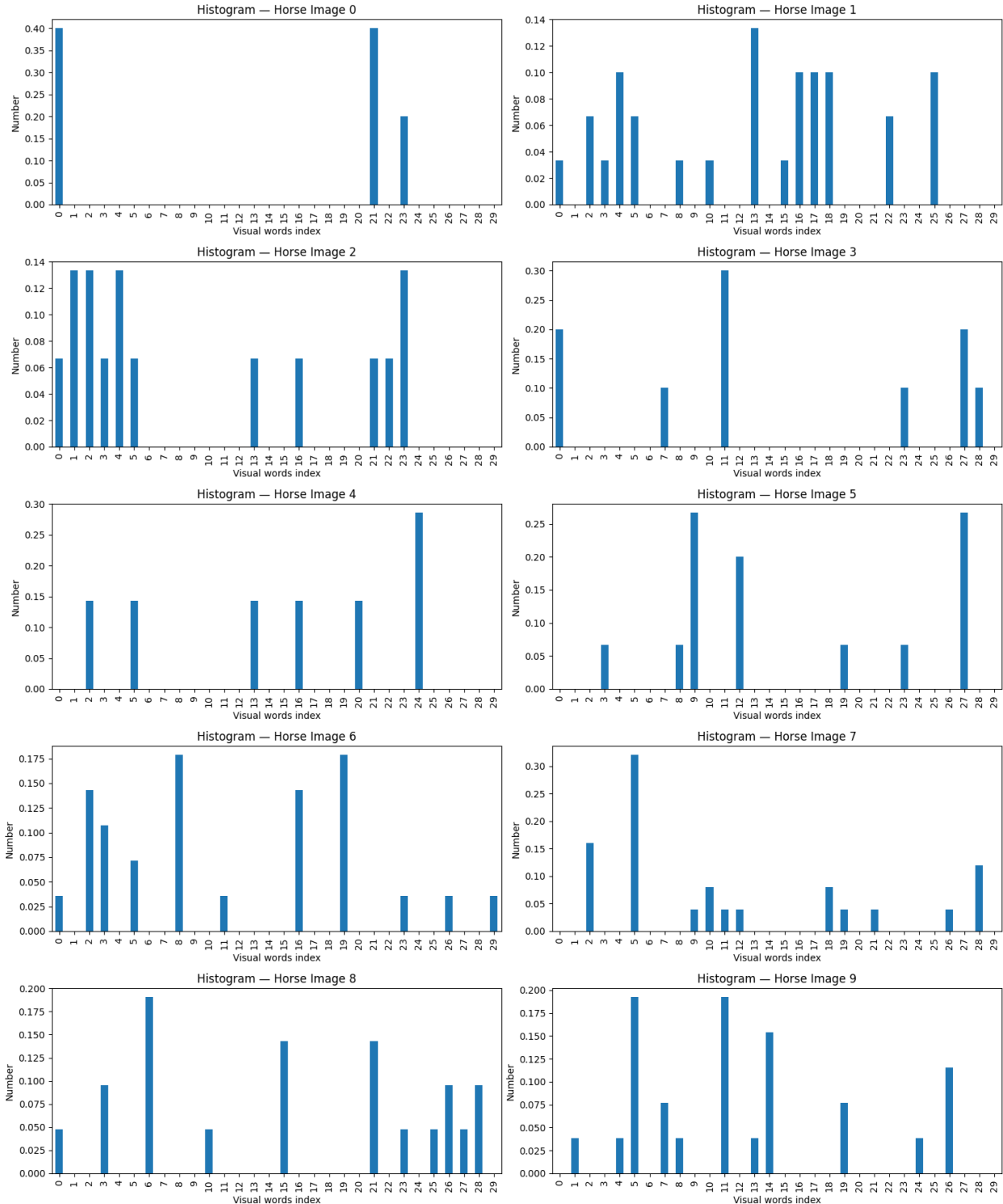
```

In this case, I went through all of the 10 horse images and applied my `featur()` function to obtain the ORB descriptors of each image. I stored the descriptors of each image in a list to be used later, but I also stored them as a whole large array. Because the descriptors may be very large, I added a check to randomly sample at most 8000 descriptors, just to make the KMeans clustering step efficient. Once this training set of descriptors was ready, I set the size of the

vocabulary to 30 visual words and then I used KMeans to cluster the descriptors into the groups. This provides me with a visual dictionary which I can now use to encode each image as a histogram of these 30 counts of visual words.

```
# Build per-image BoVW histograms
preprocessed_image = []
for d in desc_per_image:
    hist = build_histogram(d, kmeans)
    preprocessed_image.append(hist)
H = np.stack(preprocessed_image, axis=0)
# Visualizations
# Individual bar charts
fig, axes = plt.subplots(5, 2, figsize=(15, 18))
for i, hist in enumerate(preprocessed_image):
    ax = axes[i // 2, i % 2] # row, col index in 5x2 grid
    df = pd.DataFrame(hist)
    df.plot.bar(ax=ax, legend=False)
    ax.set_xlabel("Visual words index")
    ax.set_ylabel("Number")
    ax.set_title(f"Histogram - Horse Image {i}")

plt.tight_layout()
plt.show()
```



In this case, I created the Bag-of-Visual-Words Histograms of each of the 10 horse pictures and inserted them in a list, then stacked them up together into a matrix so that they could be more easily analyzed. To illustrate the results, I visualized the results as a 5×2 grid with each subplot displaying one horse image as the distribution of its visual words. I have the visual word indices (result of the KMeans clusters) on the x-axis and the counts of the repeated occurrences of each

word in that image on the y-axis. This can easily be visualized as to how the various images reveal various clustering, indicating that the horse images have their own signature, each having a unique feature.

**Question 2.** Recall that the feature scaling does not have a large effect on classification accuracy (tf-idf). How about the effect of feature scaling in Bag-of-Visual-Words? Explain it.

**Answer to Q2:** type your answer here

Scaling does not have much effect in tf-idf as the algorithm is already sensitive to differences in document length and frequency, whereas in Bag-of-Visual-Words scaling has a much more significant effect. Unless we scale, the histograms will be calculated using the raw number of descriptors, so that an image with more keypoints (such as a very detailed or noisy image) will naturally generate larger numbers than a less complex image. This may lead to a confusion by the classifier that the image is more significant simply because it has more points, rather than what the points represent. When we normalize these raw counts (typically with L1 scaling to ensure the sum of the histogram is 1), thus converting them into proportions, we thereby level the playing field in comparing two images regardless of their size, texture, or number of features identified. This allows the classifier to concentrate on the real distribution of visual words rather than being skewed by quantity. It also renders distance-based techniques such as kNN or SVM more robust, because the unscaled values in particular clusters will not control the similarity calculation. A second large benefit is that scaling dispels effects of noise or redundant features, since everything is scaled the same way. It also makes sure that even images of varying resolutions, lighting, or complexity can still be compared in a consistent manner. The results of BoVW are usually more stable and reliable with normalization than without scaling, though, in different datasets. Thing is that, whereas tf-idf does not actually require any additional scaling, BoVW generally requires it if we are interested in meaningful, consistent, and fair results.

---

**Question 3.** What is the advantage of ORB(Oriented FAST and Rotated Brief) over SURF (Speeded up robust features). Explain briefly (Minimum of 10 words and maximum of 150 words)

**Answer to Q3:** type your answer here

The greater benefit of ORB (Oriented FAST and Rotated BRIEF) compared to SURF is that it is fully open source and free, whereas SURF is patented and needs a commercial license (PDF reference). ORB is also significantly quicker and lighter, being better suited to real-time applications such as object tracking, feature matching, and mobile applications whereby computation resources are constrained. The second advantage is that ORB is the FAST keypoint detector used with the BRIEF-descriptor, but enhanced with rotation invariance and more resistant to noise, thus it remains stable in other situations. In contrast, SURF, ORB can be scaled



to a large dataset without incurring a heavy price, unlike SURF, and it is freely compatible in OpenCV with no legal limitations. In brief, ORB is not only efficient and robust but also practical and is preferred in several real world applications where speed, stability, and free usage are factored.

## Task 4. Extract text from image

Image file is given in canvas assignment .

Save it in drive and use get shareable link to everyone option . Image file name is **ExtractText4.jpeg**

Please extract the image from image file and print the text that you extracted.

```
# write your code here
import io, cv2, requests
import numpy as np
from PIL import Image
! pip install pytesseract
import pytesseract # Import pytesseract

# set path if tesseract not in PATH
# pytesseract.pytesseract.tesseract_cmd = r"C:\Program Files\
Tesseract-OCR\tesseract.exe"
!sudo apt update
!sudo apt install tesseract-ocr
pytesseract.pytesseract.tesseract_cmd = r'/usr/bin/tesseract'

LINK = "https://drive.google.com/uc?
export=download&id=1diveS93sveHl1DzrR4JGRc-qFoLW7I1l"

def fetch_image(url):
    resp = requests.get(url, stream=True)
    resp.raw.decode_content = True # Decode gzip content
    img = Image.open(resp.raw).convert("RGB")
    return cv2.cvtColor(np.array(img), cv2.COLOR_RGB2BGR)

def preprocess(img):
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
    return cv2.adaptiveThreshold(gray, 255,
cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
                                cv2.THRESH_BINARY, 31, 15)

def extract_text(img):
    return pytesseract.image_to_string(Image.fromarray(img),
config="--oem 3 --psm 6", lang="eng")

if __name__ == "__main__":
    image = fetch_image(LINK)
```

```
processed = preprocess(image)
text = extract_text(processed)
print("\n=== Extracted Text ===\n", text)
```

```
Requirement already satisfied: pytesseract in
/usr/local/lib/python3.12/dist-packages (0.3.13)
Requirement already satisfied: packaging>=21.3 in
/usr/local/lib/python3.12/dist-packages (from pytesseract) (25.0)
Requirement already satisfied: Pillow>=8.0.0 in
/usr/local/lib/python3.12/dist-packages (from pytesseract) (11.3.0)
Hit:1 https://cloud.r-project.org/bin/linux/ubuntu jammy-cran40/
InRelease
Hit:2
https://developer.download.nvidia.com/compute/cuda/repos/ubuntu2204/
x86_64 InRelease
Hit:3 https://cli.github.com/packages stable InRelease
Hit:4 http://archive.ubuntu.com/ubuntu jammy InRelease
Hit:5 http://security.ubuntu.com/ubuntu jammy-security InRelease
Hit:6 http://archive.ubuntu.com/ubuntu jammy-updates InRelease
Hit:7 http://archive.ubuntu.com/ubuntu jammy-backports InRelease
Hit:8 https://r2u.stat.illinois.edu/ubuntu jammy InRelease
Hit:9 https://ppa.launchpadcontent.net/deadsnakes/ppa/ubuntu jammy
InRelease
Hit:10 https://ppa.launchpadcontent.net/graphics-drivers/ppa/ubuntu
jammy InRelease
Hit:11 https://ppa.launchpadcontent.net/ubuntugis/ppa/ubuntu jammy
InRelease
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
40 packages can be upgraded. Run 'apt list --upgradable' to see them.
W: Skipping acquire of configured file 'main/source/Sources' as
repository 'https://r2u.stat.illinois.edu/ubuntu jammy InRelease' does
not seem to provide it (sources.list entry misspelt?)
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
tesseract-ocr is already the newest version (4.1.1-2.1build1).
0 upgraded, 0 newly installed, 0 to remove and 40 not upgraded.
```

=== Extracted Text ===

```
"c
Education is the
passport to the future,
for tomorrow belongs
to those who prepare
for it today.
Malcolm X
```

In this code, I perform text extraction (OCR) from an image using Tesseract. First, I import the necessary libraries like OpenCV, NumPy, Pillow, and pytesseract. I also install Tesseract on the system to ensure it is available. I provide a Google Drive link for the input image and define a helper function, `fetch_image()`, which downloads the image from the link, opens it with Pillow, and converts it into a format compatible with OpenCV. Next, I did preprocess the image using the function `preprocess()` thus by converting it to grayscale and applying adaptive thresholding. This step helps improve the clarity of the text against the background. Finally, in the `extract_text()` function, I use pytesseract's `image_to_string()` method with OCR settings to read the text. Next in the main block, I did combine all these steps: first being fetching the image, secondly preprocessing it, then extracting the text, and then finally printing the extracted clean text in a readable format. This process ensures that even noisy or poorly lit images are optimized for accurate OCR results.

## Task 5. Design code to implement **ORB** using **OpenCV**. Please use a face image (Preferably your face image as its unique).

Keep your face image file name as **faceimg.jpg** Save it in drive and use get shareable link to everyone option.

Please print the following during the implementation

- Training image and Testing image
- Number of Keypoints Detected In The Training Image (Not just number image too)
- Number of Keypoints Detected In The Query Image (Not just number image too)
- Number of Matching Keypoints Between The Training and Query Images (Not just number image too)

```
import io, re, requests, numpy as np, cv2, matplotlib.pyplot as plt
from PIL import Image

def fetch_image(url):
    r = requests.get(url, stream=True, timeout=60);
    r.raise_for_status()
    return
cv2.cvtColor(np.array(Image.open(io.BytesIO(r.content))).convert("RGB"), cv2.COLOR_RGB2BGR)

def get_direct_drive_link(share_link):
    file_id = share_link.split("/")[-2]
    return f"https://drive.google.com/uc?export=download&id={file_id}"

# Replace with your links
train_link =
get_direct_drive_link("https://drive.google.com/file/d/1lflwGkMUDYWjq4
ux9h8H9UVfEkprSONk/view?usp=drive_link")
```

```

test_link =
get_direct_drive_link("https://drive.google.com/file/d/1MgK0rfVsamje4c
XyGRgC3EpZxb9WtIno/view?usp=drive_link")

# --- Load images ---
img_train = fetch_image(train_link)
img_test = fetch_image(test_link)

# --- Resize test image to match train image size ---
h, w = img_train.shape[:2]
img_test = cv2.resize(img_test, (w, h))

print("Training image:")
plt.imshow(cv2.cvtColor(img_train, cv2.COLOR_BGR2RGB));
plt.axis('off'); plt.show()
print("\nTesting image:")
plt.imshow(cv2.cvtColor(img_test, cv2.COLOR_BGR2RGB));
plt.axis('off'); plt.show()

# --- ORB ---
orb = cv2.ORB_create(nfeatures=1000)
gray_train = cv2.cvtColor(img_train, cv2.COLOR_BGR2GRAY)
gray_test = cv2.cvtColor(img_test, cv2.COLOR_BGR2GRAY)
kp_train, des_train = orb.detectAndCompute(gray_train, None)
kp_test, des_test = orb.detectAndCompute(gray_test, None)

print(f"Number of Keypoints Detected In The Training Image:
{len(kp_train)}")
print(f"Number of Keypoints Detected In The Query Image:
{len(kp_test)}")

# --- Draw keypoints ---
img_train_kp = cv2.drawKeypoints(img_train, kp_train, None,
color=(0,255,0), flags=0)
img_test_kp = cv2.drawKeypoints(img_test, kp_test, None,
color=(0,255,0), flags=0)
print("\nTraining image with keypoints:")
plt.imshow(cv2.cvtColor(img_train_kp, cv2.COLOR_BGR2RGB));
plt.axis('off'); plt.show()
print("\nTesting image with keypoints:")
plt.imshow(cv2.cvtColor(img_test_kp, cv2.COLOR_BGR2RGB));
plt.axis('off'); plt.show()

# --- Match ---
if des_train is not None and des_test is not None:
    bf = cv2.BFMatcher(cv2.NORM_HAMMING, crossCheck=True)
    matches = bf.match(des_train, des_test)
    matches = sorted(matches, key=lambda x: x.distance)
    print(f"\nNumber of Matching Keypoints: {len(matches)}")

```

```
# Show top 50 matches
img_matches = cv2.drawMatches(img_train, kp_train, img_test,
                               kp_test,
                               matches[:50], None,
                               flags=cv2.DrawMatchesFlags_NOT_DRAW_SINGLE_POINTS)
plt.figure(figsize=(15, 10))
plt.imshow(cv2.cvtColor(img_matches, cv2.COLOR_BGR2RGB))
plt.title(f"Matching Keypoints (Top {min(50, len(matches))})")
plt.axis('off')
plt.show()
else:
    print("\nDescriptors not found. Try higher resolution or clearer
    face images.")
Training image:
```

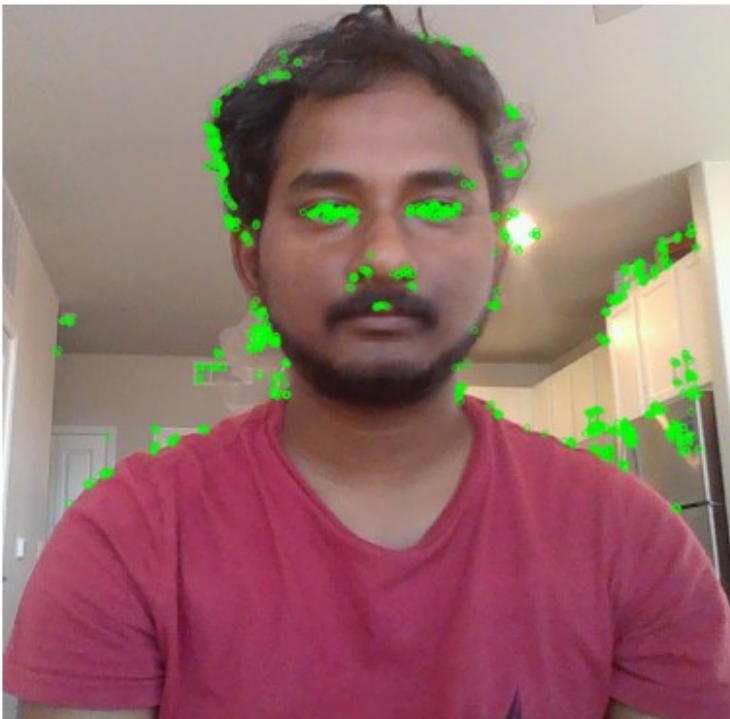


Testing image:



Number of Keypoints Detected In The Training Image: 996  
Number of Keypoints Detected In The Query Image: 971

Training image with keypoints:



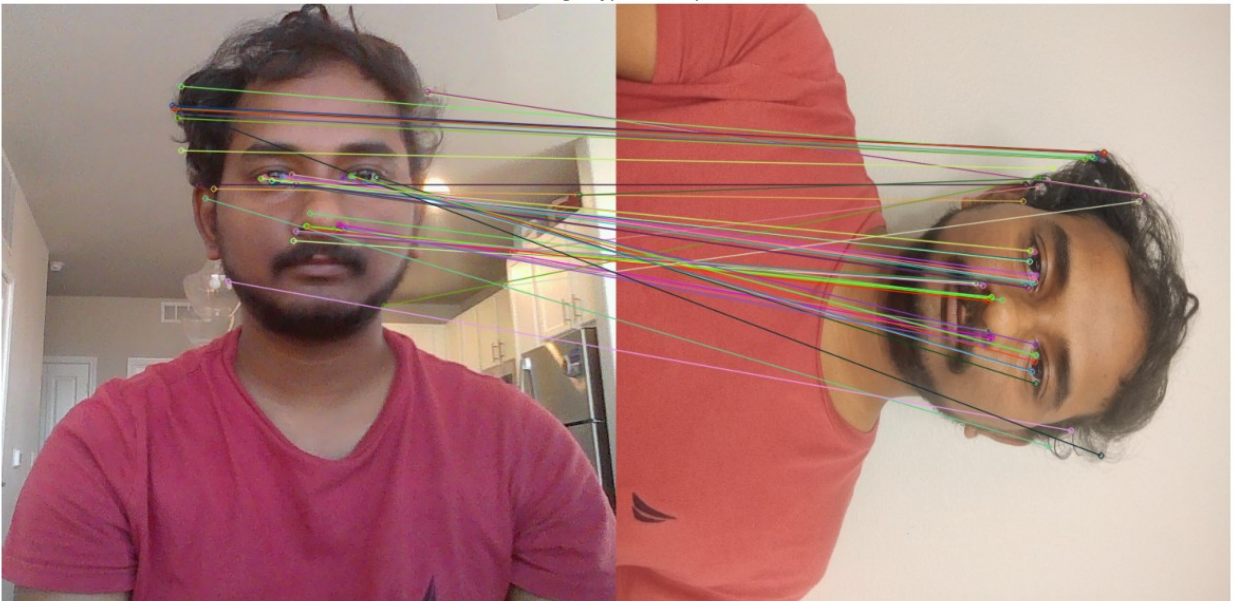


Testing image with keypoints:



Number of Matching Keypoints: 250

Matching Keypoints (Top 50)



In this code, I perform feature detection and matching between two face images using ORB in OpenCV. First, I created helper functions to fetch images directly from Google Drive links and convert them into a format compatible with OpenCV. After loading both the training and testing images, I resized the test image so they have the same dimensions, which makes the comparison cleaner. Then, I converted them to grayscale and used the ORB detector to find up to 1000 keypoints and compute descriptors. I printed the number of keypoints detected in both images and visualized them by drawing the detected points on each image. For matching, I used the brute-force matcher with the Hamming distance and enabled cross-checking to ensure reliable pairings. I sorted the matches based on distance and printed the total number of matching keypoints before finally visualizing the top 50 matches side by side. This complete workflow shows how ORB can effectively identify and compare unique feature points across two images.

Reference: For this section i refered the chatgpt, with the prompt, give code to resize the images and the function fecth url as the direct url is giving error saying file not found.